

TECHNICAL REPORT

MealPlanner v2

Ingredient Resource Plan

An ingredient-centered resource map for moving the current
localStorage React planner into a server-client architecture.

Chrono-chan

Version 1.2 · 2026.05.10

Prepared for SirEdvin · MealPlanner

| Contents

01	Executive Summary	03
02	Current Application Shape	04
03	Resource Model	05
04	Endpoint Contract Sketch	08
05	Implementation Sequence	10
06	Appendix	11

01 · EXECUTIVE SUMMARY

Executive Summary

MealPlanner v2 should treat the current React planner as a useful prototype, not as the server data model. The corrected core REST shape should be [MealPlan](#) → [MealCell](#) → [Ingredient](#) links, where each weekly cell references multiple ingredients. Ingredients may be raw single foods or combined prepared items, and they preserve the original planner's type metadata.

Key Takeaways

- The current app stores everything in one browser-local object: weeks, active week, bank, theme, and design mode.
- The server should not expose the current `cells["0-m1"]` object as the canonical model; use dated meal cells and slots instead.
- The practical MVP resource set is `/meal-plans` , `/meal-cells` , `/ingredients` , `/ingredient-types` , `/summary` , and `/imports/text` .
- Households and eaters are worth adding early, because the domain is family-centered and baby-feeding-specific.

RECOMMENDED MVP

Start with six persistent concepts: Household, Eater, MealPlan, MealCell, Ingredient, and IngredientType. Model ingredient kinds explicitly: raw ingredient versus combined ingredient.

Current Application Shape

The inspected app is a Vite React planner with one large `App.jsx`, CSS-driven print layouts, import/export, a reusable ingredient bank, and localStorage persistence under the key `meal-planner-v2`.

Observed Client Model

Current concept	Where it lives	Server-side interpretation
Weeks	<code>store.weeks[]</code>	Meal plans, preferably date-ranged and owned by a household/eater.
Cells	<code>week.cells[day-slot]</code>	Meal cells with date/day, slot, and ordered links to multiple ingredients.
Ingredient bank	<code>store.bank[]</code>	Reusable raw or combined ingredients scoped to a household.
Ingredient types	<code>allergen</code> , <code>cook</code> , <code>prep</code>	Original planner type metadata; preserve as typed ingredient classifications.
Theme / design	<code>store.theme</code> , <code>store.designMode</code>	User preferences plus optional per-plan presentation overrides.

The Core Boundary

The frontend may keep a grid-friendly shape for rendering, but the API should expose stable resources: plans, cells, ingredients, ingredient links, and ingredient types. UI convenience should not become database architecture.

Current Feature Inventory

- Planning: 7-day plans with 1-3 meal slots per day.
- Editing: Click-to-edit meal cells with bank suggestions and chip-like items.
- Ingredient bank: Saved ingredient names with original type flags: allergen, cook, and prep.
- Import/export: Text import for day lists and JSON import/export for week/all data.
- Output: A4 print/PDF layout with multiple design modes.
- Summary: Derived weekly lists from ingredient links and ingredient types: buy, cook, prep, and allergens.

03 · RESOURCE MODEL

Resource Model

The REST model should be ingredient-centered and boring in the good way: nouns for persistent things, nested routes only when ownership is meaningful, and derived views separated from mutable resources.

Primary Resources

Resource	Purpose	Priority
<code>/me</code>	Current account identity and default preferences.	MVP if auth exists
<code>/households</code>	Family/container ownership for plans, eaters, and ingredient bank.	MVP
<code>/eaters</code>	Baby/child/person profile with restrictions, age, notes, and preferences.	MVP
<code>/meal-plans</code>	Week-like planning document containing dated meal cells.	MVP core
<code>/meal-slots</code>	Named slots such as morning, midday, evening.	Embed or MVP-lite
<code>/meal-cells</code>	One cell in a weekly plan: day/date + slot + ordered ingredient links.	MVP core
<code>/ingredients</code>	Reusable ingredient bank. Each ingredient is either <code>raw</code> (tomato) or <code>combined</code> (tomato and potato salad).	MVP core
<code>/ingredient-types</code>	Original planner classifications such as allergen, needs cooking, and prep item.	MVP core

Derived and Processing Resources

Resource	Purpose	REST behavior
<code>/meal-plans/{id}/summary</code>	Buy/cook/prep/allergen rollup.	Read-only, computed from meal-cell ingredient links and ingredient types.
<code>/meal-plans/{id}/imports/text</code>	Convert pasted day-list text into cells.	Processing endpoint; optionally preview first.
<code>/meal-plans/{id}/export</code>	Download plan data as JSON, later PDF.	Read-only representation endpoint.
<code>/ingredient-types</code>	Expose built-in ingredient type definitions.	Read-only until user-custom types exist.

Canonical Data Shape

The main migration is from anonymous cell keys to explicit meal cells. A current key like `0-m1` maps to a date/day index and a slot ID. The cell itself does not own free text; it owns ordered links to ingredient records.

```
{
  "id": "cell_123",
  "planId": "plan_123",
  "date": "2026-05-03",
  "dayIndex": 0,
  "slotId": "slot_morning",
  "ingredients": [
    { "ingredientId": "ing_1", "nameSnapshot": "Tomato", "position": 1 },
    { "ingredientId": "ing_2", "nameSnapshot": "Tomato and potato salad",
      "position": 2 }
  ]
}
```

Ingredient Model

The ingredient resource is the center of the new model. It covers both raw ingredients and combined prepared ingredients without losing the original planner's simple bank behavior.

```
{
  "id": "ing_2",
  "householdId": "house_123",
  "name": "Tomato and potato salad",
  "kind": "combined",
  "componentIngredientIds": ["ing_tomato", "ing_potato"],
  "typeIds": ["prep"],
  "notes": "Prepared component for multiple cells"
}

{
  "id": "ing_tomato",
  "householdId": "house_123",
  "name": "Tomato",
  "kind": "raw",
  "componentIngredientIds": [],
  "typeIds": [],
  "notes": null
}
```

IMPORTANT CORRECTION

A combined ingredient is still an ingredient. It may optionally link to component raw ingredients for shopping and preparation rollups, but the weekly meal plan cell links to the combined ingredient as one selectable item. This keeps the UI simple while giving the backend enough structure for summaries.

Why Ingredients, Not Generic Food Items

The term **ingredient** is more precise for MealPlanner v2 because the planner is assembling meal cells from reusable inputs. The model should allow raw ingredients such as tomato and combined ingredients such as tomato and potato salad. This avoids a vague resource allocation layer while preserving the original simple picker experience.

04 · ENDPOINT CONTRACT

| Endpoint Contract Sketch

The endpoint design should favor predictable CRUD for persistent nouns, plus a few scoped action-like processing endpoints for imports, exports, and summaries.

Recommended Endpoint Map

```
# Account and family
GET    /me
PATCH /me
GET    /households
POST   /households
GET    /households/{householdId}
PATCH /households/{householdId}

# Eaters / child profiles
GET    /households/{householdId}/eaters
POST   /households/{householdId}/eaters
GET    /eaters/{eaterId}
PATCH /eaters/{eaterId}
DELETE /eaters/{eaterId}

# Plans
GET    /meal-plans?householdId=...&eaterId=...
POST   /meal-plans
GET    /meal-plans/{planId}
PATCH /meal-plans/{planId}
DELETE /meal-plans/{planId}

# Meal cells / weekly grid
GET    /meal-plans/{planId}/cells
POST   /meal-plans/{planId}/cells
PUT    /meal-plans/{planId}/cells/{date}/{slotId}
PATCH /meal-cells/{cellId}
DELETE /meal-cells/{cellId}

# Ingredient bank
GET    /ingredients?householdId=...&q=...&kind=raw|combined
POST   /ingredients
GET    /ingredients/{ingredientId}
PATCH /ingredients/{ingredientId}
DELETE /ingredients/{ingredientId}
GET    /ingredients/{ingredientId}/components
PUT    /ingredients/{ingredientId}/components

# Ingredient type metadata
GET    /ingredient-types
POST   /ingredient-types      # optional after MVP if custom types are allowed
PATCH /ingredient-types/{typeId}

# Derived and processing
GET    /meal-plans/{planId}/summary
POST   /meal-plans/{planId}/imports/text/preview
POST   /meal-plans/{planId}/imports/text
GET    /meal-plans/{planId}/export?format=json
```

Resource Ownership Rules

Resource	Owner	Reason
Ingredient	Household	The ingredient bank is shared by family context, not one plan.
Meal plan	Household + eater	A plan is usually for a specific child/person.
Meal cell	Meal plan	Cells do not make sense outside the plan grid.
Preferences	User, with plan override	Defaults are personal; print appearance may be plan-specific.

Update Semantics

Use `PUT /meal-plans/{planId}/cells/{date}/{slotId}` for replacing one cell. This matches the UI action exactly: the user edits a weekly cell, and the client sends the new ordered ingredient-link list. Use `PATCH /meal-cells/{cellId}` for partial changes such as notes or item order once cells have stable IDs.

05 · IMPLEMENTATION SEQUENCE

Implementation Sequence

Build the backend-backed planner in thin vertical slices. The first slice should sync one plan and one ingredient bank reliably before adding extra social, print, or import sophistication.

Phase Plan

1. Contract first: Write an OpenAPI skeleton for households, eaters, ingredients, ingredient types, meal plans, meal cells, and summary.
2. Persistence: Add database models and migrations for the ingredient-centered MVP entities.
3. Ingredient bank API: Implement create/search/update/delete for `ingredients`, including `kind`, component links, and type arrays.
4. Plan API: Implement `meal-plans` and meal-cell replacement by date/slot.
5. Client adapter: Convert REST cells to the current grid shape and back inside one boundary module.
6. Summary: Reproduce the current weekly summary server-side and keep client calculation as a temporary fallback.
7. Imports: Add text import preview, then save parsed cells and ingredient links.

Client Migration Boundary

Keep the current React components as much as possible, but add a data adapter layer with two functions: `mealCellsToGrid(cells)` and `cellUpdateToIngredientLinksPayload(date, slotId, ingredients)`. This localizes the migration and prevents the UI from dictating the API.

FIRST CODING TARGET

Implement `ingredients`, `ingredient-types`, `meal-plans`, and `PUT /meal-plans/{id}/cells/{date}/{slotId}` before anything else. If these work, the existing UI can become a synced planner without redesigning every component.

Appendix

A. Suggested Database Entities

User
 Household
 HouseholdMember
 Eater
 MealPlan
 MealSlot
 MealCell
 MealCellIngredient
 Ingredient
 IngredientComponent
 IngredientType
 IngredientTypeAssignment
 UserPreference

B. MVP Schema Notes

Table	Important fields
meal_plans	id , household_id , eater_id , name , start_date , end_date , meal_slot_count , theme , design_mode
meal_cells	id , meal_plan_id , date , day_index , slot_id , notes
meal_cell_ingredients	id , meal_cell_id , ingredient_id , name_snapshot , position
ingredients	id , household_id , name , normalized_name , kind (raw or combined) , emoji , notes
ingredient_components	parent_ingredient_id , component_ingredient_id , quantity , unit , position
ingredient_types	id , label , short_label , description , built_in
ingredient_type_assignments	ingredient_id , type_id

C. Evidence Trail

- Inspected current web planner source: `/home/siredvin/projects/MealPlanner/src/App.jsx` .
- Observed Vite/React dependencies: `/home/siredvin/projects/MealPlanner/package.json` .
- Observed persisted localStorage key: `meal-planner-v2` .
- Observed current features in source: weeks, cell grid, bank, ingredient type toggles (`allergen` , `cook` , `prep`), text import, JSON import/export, summary modal, print/PDF button, and design modes.

- Note: recalled context mentions a larger MealPlanner monorepo under `/opt/Projects/MealPlanner` , but that path was not present in this session. The report therefore grounds its concrete evidence in the available web planner path.

D. Glossary

Meal plan - A week-like or date-ranged planning document.

Meal cell - One cell of the plan: a specific date/day and slot containing ordered ingredient links.

Ingredient - A reusable bank item. It can be raw, such as tomato, or combined, such as tomato and potato salad.

Ingredient type - A planner classification preserved from the original app, currently allergen, cook, and prep.